Toxic Comment Classification

A Multilabel Deep Learning Approach

NLP · CNN-GRU Hybrid · Weighted Binary Cross-Entropy

Table of Contents

1. Introduction & Problem Statement

Online platforms face a growing challenge: moderating user-generated content at scale. Toxic comments — those that are rude, disrespectful, or likely to make someone leave a discussion — undermine healthy online conversations and can cause real psychological harm. Manual moderation is neither scalable nor consistent, making automated detection essential.

This project addresses this challenge by building a **multilabel text classification system** capable of simultaneously detecting six distinct categories of toxicity in a single comment: **toxic**, **severe_toxic**, **obscene**, **threat**, **insult**, and **identity_hate**. Unlike multiclass classification (where each input maps to exactly one class), multilabel classification allows a comment to belong to zero, one, or multiple categories at the same time — for example, a comment can be simultaneously *toxic*, *obscene*, and an *insult*.

The project follows a structured machine-learning pipeline: exploratory data analysis, text preprocessing, feature engineering, baseline modeling with classical ML, and progressive deep-learning experimentation culminating in a CNN-GRU hybrid architecture trained with a custom weighted loss function.

2. Dataset Overview

The dataset used in this project is the **Filter Toxic Comments dataset**, loaded from an S3 bucket. It contains **159,571 observations** and **8 columns**:

- **comment_text**: the raw text of the user comment.
- **toxic**: binary flag (0/1) indicating general toxicity.
- **severe_toxic**: binary flag for severely toxic content.
- **obscene**: binary flag for obscene language.
- **threat**: binary flag for threatening content.
- **insult**: binary flag for insults.
- **identity_hate**: binary flag for hate speech targeting identity groups.
- **sum_injurious**: the sum of the six binary labels above (range 0–6).

Each comment can carry multiple labels simultaneously, making this a multilabel problem. The *sum_injurious* column aggregates the total number of toxic categories assigned to each comment, where a value of 0 means the comment is clean.

Initial checks confirmed the dataset contains **no missing values** and **no duplicate rows**, so no imputation or deduplication was needed before proceeding.

3. Exploratory Data Analysis

3.1 Class Imbalance

The most striking characteristic of this dataset is the severe class imbalance. Out of 159,571 comments, approximately **89.8% are clean** (sum_injurious = 0) and only **10.2% contain any form of toxicity**. This imbalance is a critical consideration: a naive model that always predicts 'clean' would achieve ~90% accuracy while being completely useless for the actual task of detecting toxic content.

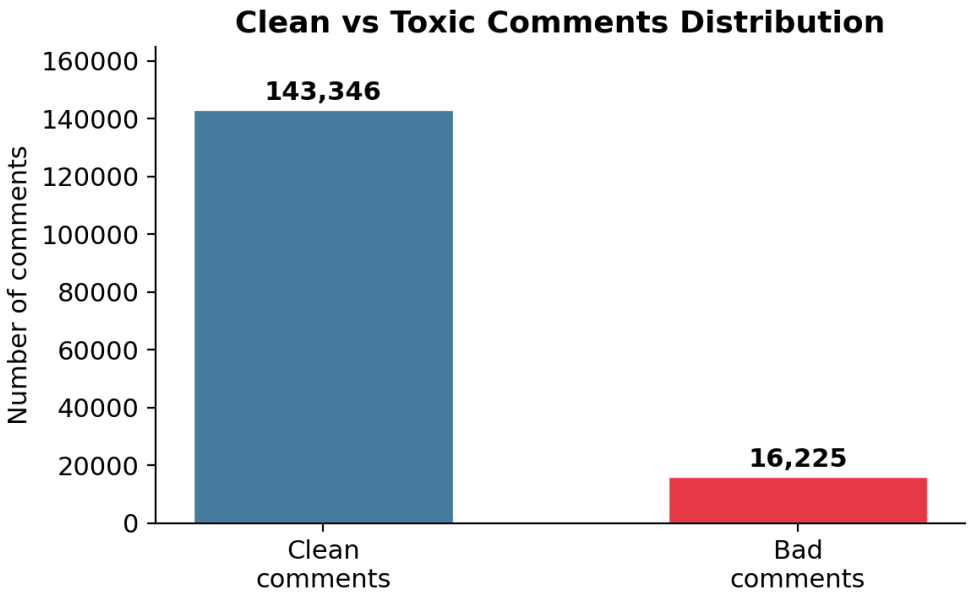

Figure 1: Distribution of clean vs toxic comments in the dataset.

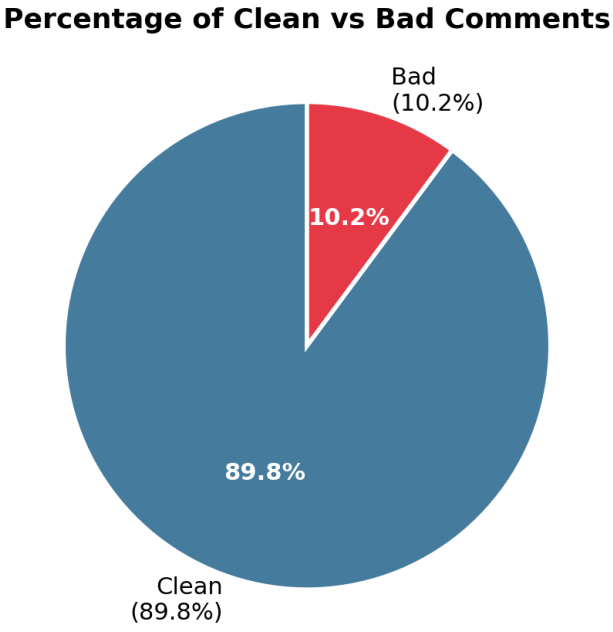

Figure 2: Percentage breakdown of clean vs toxic comments.

3.2 Per-Category Distribution

Within the toxic comments, the distribution across the six categories is also highly uneven. **toxic** is the most common label (15,294 occurrences), followed by **obscene** (8,449) and **insult** (7,877). The rarest categories are **severe_toxic** (1,595), **identity_hate** (1,405), and **threat** (only 478 occurrences). This means the model must handle a 32:1 ratio

between the most and least frequent categories.

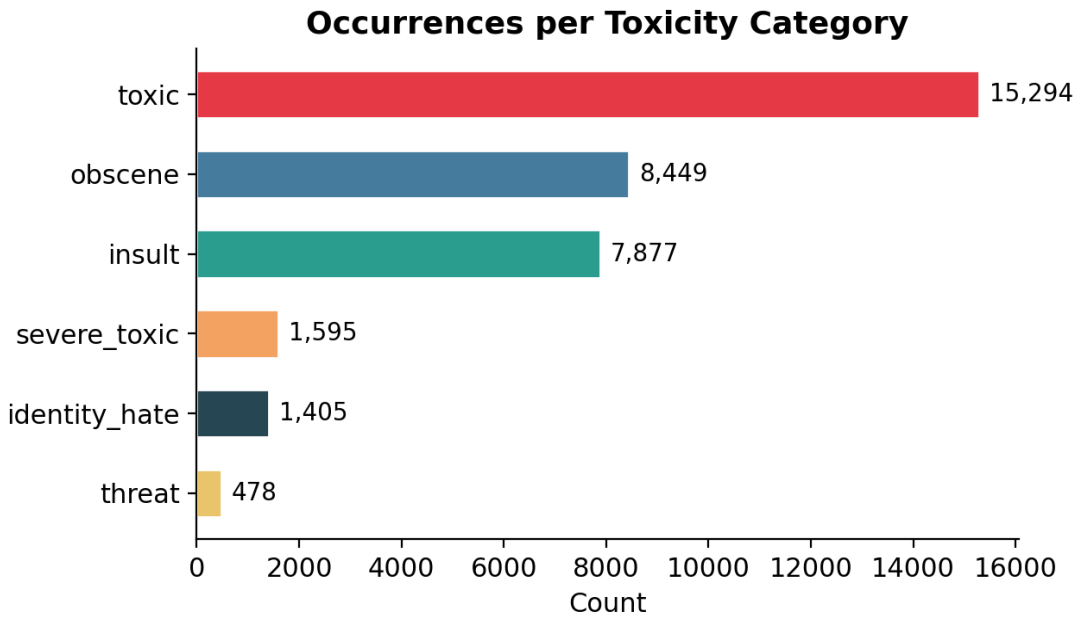

Figure 3: Occurrence count for each toxicity category.

3.3 Label Correlation Analysis

Understanding how the labels co-occur is important for multilabel classification. The correlation matrix below reveals several noteworthy patterns:

- **obscene** and **insult** are the most correlated pair ($r \approx 0.74$), suggesting that obscene comments very often contain insults.
- **toxic** correlates strongly with both **obscene** ($r \approx 0.68$) and **insult** ($r \approx 0.65$), which makes sense: most obscene or insulting content is also broadly toxic.
- **threat** has the weakest correlations with all other labels, indicating that threatening comments form a somewhat independent category.

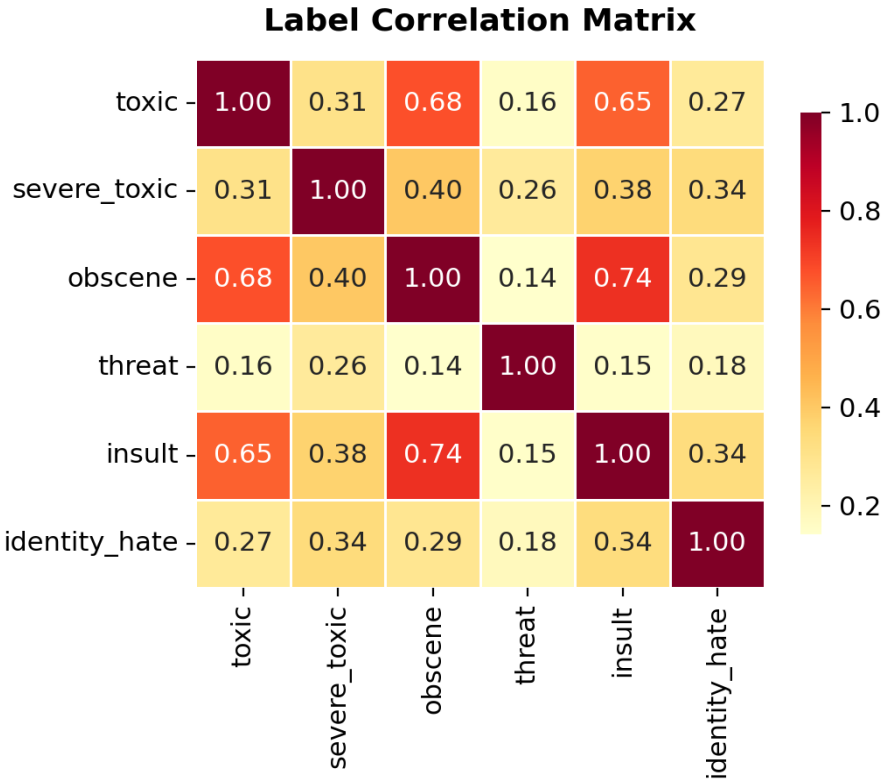

Figure 4: Pairwise Pearson correlation between the six toxicity labels.

3.4 Comment Length Analysis

The length of comments (measured in characters) provides useful insight into the nature of the data. Most comments are relatively short (under 500 characters), but there is a long right tail with some comments exceeding several thousand characters. Importantly, toxic comments tend to be **longer on average** than clean comments, and comments with higher sum_injurious values (more simultaneous toxicity categories) tend to be longer still. This suggests that the sheer volume of text may correlate with the intensity or multi-dimensionality of toxic content.

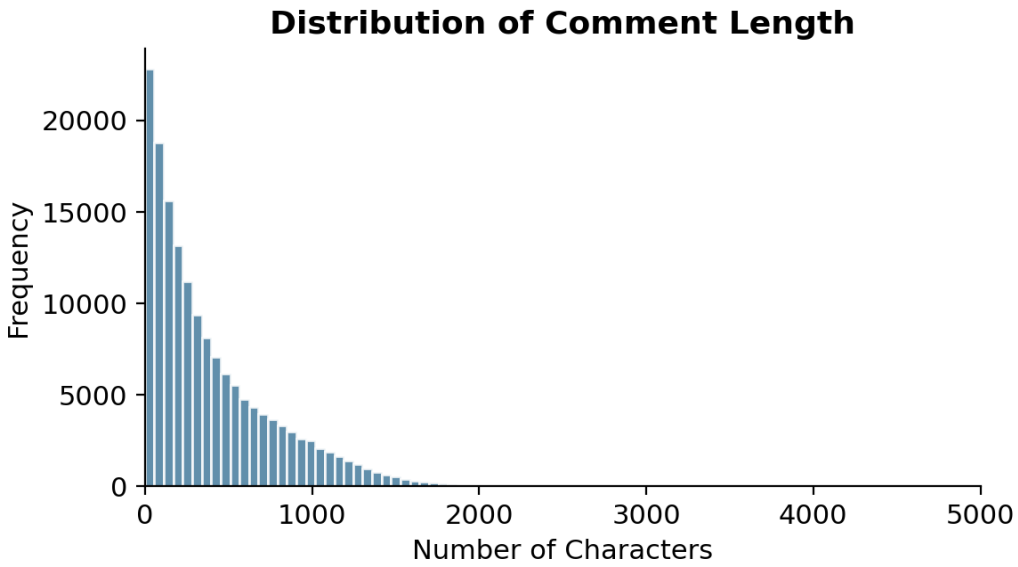

Figure 5: Histogram of comment length (character count).

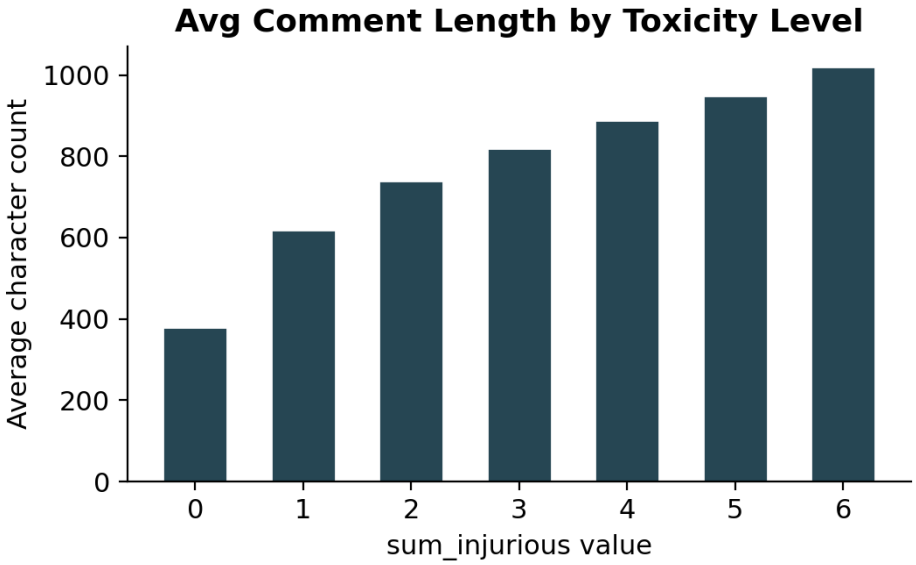

Figure 6: Average comment length grouped by sum_injurious value.

Word clouds were also generated for each toxicity category to visualize the most frequent terms. Clean comments showed general-purpose vocabulary (article, page, talk, Wikipedia), while toxic categories revealed domain-specific offensive language, slurs, and aggressive terms. The **threat** word cloud contained words related to violence and death, while **identity_hate** surfaced slurs and derogatory terms targeting specific demographic groups.

4. Data Preprocessing

4.1 Undersampling Strategy

To mitigate the severe class imbalance (89.8% clean vs 10.2% toxic), an undersampling strategy was applied to the majority class. Specifically, only **one third of the clean comments** were retained (randomly sampled with random_state=42 for reproducibility), while **all toxic comments were kept**. This reduced the clean-comment count from 143,346 to approximately 47,782, yielding a more balanced dataset where toxic comments represent a larger proportion. The final undersampled dataset was shuffled to ensure uniform mixing of clean and toxic examples.

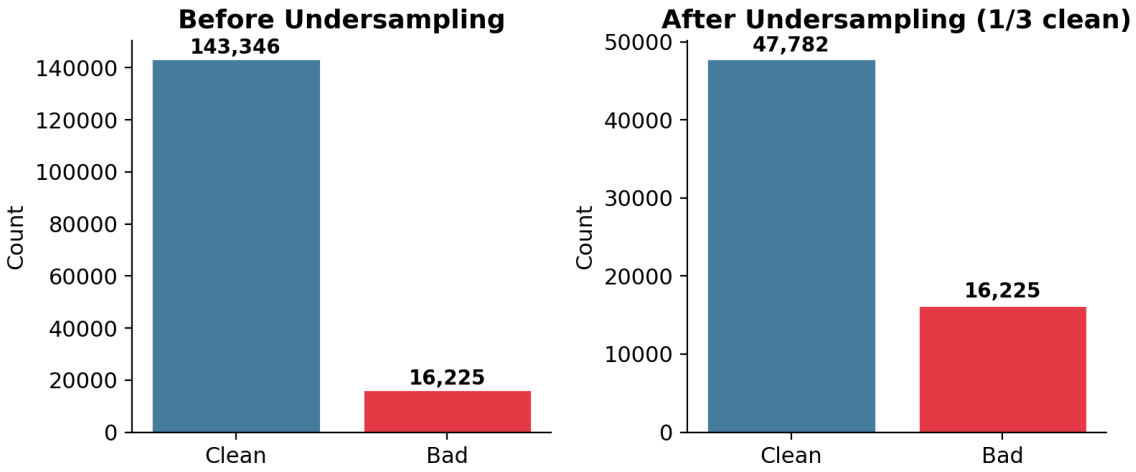

Figure 7: Effect of undersampling on class distribution.

4.2 Text Cleaning Pipeline

Raw comment text requires extensive cleaning before it can be fed into a machine-learning model. The preprocessing pipeline applies four sequential transformations:

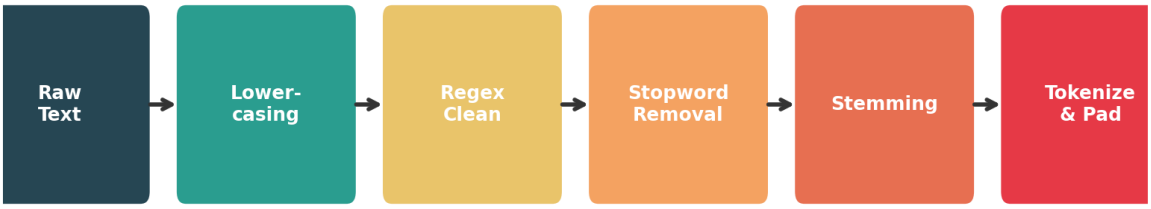

Figure 8: Text preprocessing pipeline.

Step 1 — Lowercasing: All text is converted to lowercase to ensure that 'Toxic', 'TOXIC', and 'toxic' are treated as the same word. This reduces vocabulary size without losing semantic meaning.

Step 2 — Regex Cleaning: A regular expression removes words that consist of a single letter surrounded by punctuation, tabs, or line breaks. This eliminates fragments and artifacts that carry no meaning (e.g., stray characters from formatting).

Step 3 — Stopword Removal: Common English stopwords (the, is, and, etc.) are removed using the NLTK stopword list. These high-frequency words appear in virtually all comments regardless of toxicity and would add noise rather than signal to the model.

Step 4 — Stemming: The Snowball Stemmer reduces each remaining word to its root form (e.g., 'running' → 'run', 'hateful' → 'hate'). This further reduces vocabulary size and helps the model generalize across different inflected forms of the same word.

The preprocessed data was saved as a Parquet file to avoid repeating this computationally expensive step on every run.

5. Tokenization & Sequence Preparation

5.1 Train / Validation / Test Split

The preprocessed dataset was split into three subsets using a custom *train_test_val_split* function that wraps scikit-learn's *train_test_split*. The split ratios were: **64% training**, **16% validation**, and **20% test**. The validation set is used for hyperparameter tuning and early stopping, while the test set provides a final, unbiased evaluation of each model.

5.2 Tokenization and Padding

The cleaned text was converted into numerical sequences using Keras' **Tokenizer** class with a vocabulary size of 1,000 (the top 1,000 most frequent words in the training set). Words outside this vocabulary are replaced with an out-of-vocabulary token.

Since neural networks require fixed-length inputs, all sequences were padded (or truncated) to the length of the longest sequence in the training set using Keras' **pad_sequences** function. Shorter sequences are zero-padded at the beginning (pre-padding), which is the default and works well with recurrent architectures that process sequences from left to right.

The *vocab_size* and *max_len* values returned by this step are critical hyperparameters for the Embedding layer and the input shape of all subsequent models.

6. Baseline Models

Before diving into deep learning, two classical machine-learning baselines were trained to establish a performance floor. Both used **TF-IDF** (Term Frequency–Inverse Document Frequency) vectorization to convert text into numerical features, and **OneVsRestClassifier** to adapt binary classifiers to the multilabel setting.

6.1 Logistic Regression

A Logistic Regression model was trained with *class_weight='balanced'* to automatically adjust weights inversely proportional to class frequencies. A second approach computed explicit per-class weights based on label frequencies and trained separate logistic models for each label.

Results showed reasonable precision but poor recall for minority classes, and noticeable overfitting (training metrics significantly better than test metrics). The model struggled particularly with **threat** and **identity_hate**, the rarest categories.

6.2 Naive Bayes

Three Naive Bayes variants were evaluated using GridSearchCV over the smoothing parameter α: **MultinomialNB** (best $\alpha = 0.01$) and **ComplementNB** (best $\alpha = 0.5$). ComplementNB is specifically designed for imbalanced datasets, as it estimates parameters from the complement of each class.

Despite the theoretical advantages of ComplementNB for imbalanced data, both Naive Bayes models performed worse than Logistic Regression. MultinomialNB suffered from severe overfitting, and ComplementNB produced even lower metrics overall.

Conclusion: Logistic Regression outperformed Naive Bayes, but neither baseline achieved satisfactory results — especially for recall on minority classes. This motivated the transition to recurrent neural networks, which can capture sequential patterns in text.

7. Custom Loss Function & Evaluation Metrics

7.1 Hamming Loss

For multilabel classification, standard accuracy is misleading because it treats the entire label vector as a single prediction. Instead, **Hamming Loss** is used, defined as the fraction of incorrectly predicted labels across all samples and all classes:

$$HL = (1 / N \cdot L) \cdot \Sigma_i \Sigma_j \blacksquare(y_{ij} \neq \blacksquare_{ij})$$

where N is the number of samples, L is the number of labels, y_{ij} is the true label, and $\blacksquare_{ij}$ is the predicted label. A lower Hamming Loss indicates better performance. However, because Hamming Loss is not differentiable, it is used only as a monitoring metric during training, not as the training loss function.

7.2 Weighted Binary Cross-Entropy

The actual training loss is a **custom weighted binary cross-entropy**, designed to counteract class imbalance by assigning higher weights to underrepresented labels. For each label j, two weights are computed:

- $\mathbf{w_{1,j}}$ (weight for positive class): proportional to N / count(label j = 1)
- $\mathbf{w_{0,j}}$ (weight for negative class): proportional to N / count(label j = 0)

These weights are normalized so they sum to 1. The effect is that misclassifying a rare label (e.g., *threat* with only 478 positive examples) incurs a much larger penalty than misclassifying a common label, forcing the model to pay attention to minority classes.

7.3 Early Stopping

All deep-learning models use the **EarlyStopping** callback monitoring *val_loss* with a patience of 4 epochs and *restore_best_weights=True*. This means training halts if the validation loss does not improve for 4 consecutive epochs, and the model reverts to the weights from the best epoch. This prevents overfitting and avoids wasting computational resources on unproductive training.

8. Deep Learning Models

Five deep-learning architectures were explored in sequence, each building on insights from the previous one. All models share the same training setup: weighted binary cross-entropy loss, Hamming Loss as a monitoring metric, RMSprop or Adam optimizer, batch size of 64, and EarlyStopping with patience 4.

8.1 Weighted SimpleRNN

The first neural model uses a **SimpleRNN** layer — the most basic form of recurrent neural network. The architecture is: Embedding (vocab_size → 128d) → Dropout (0.8) → SimpleRNN (64 units, tanh) → Dropout (0.5) → Dense (6, sigmoid). The high dropout rates (0.8 and 0.5) were chosen to aggressively combat overfitting.

Results showed a significant improvement in **recall** compared to the logistic regression baseline, meaning the model catches more actual toxic comments. However, **precision dropped substantially**, indicating many false positives — the model was over-predicting toxicity. The SimpleRNN also suffers from the vanishing gradient problem, limiting its ability to capture long-range dependencies.

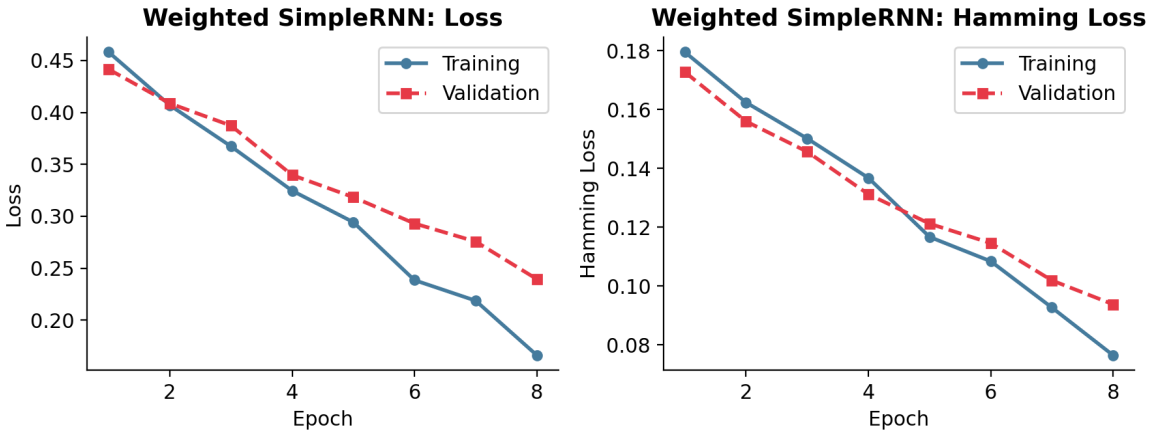

Figure 9: Learning curves for the Weighted SimpleRNN model.

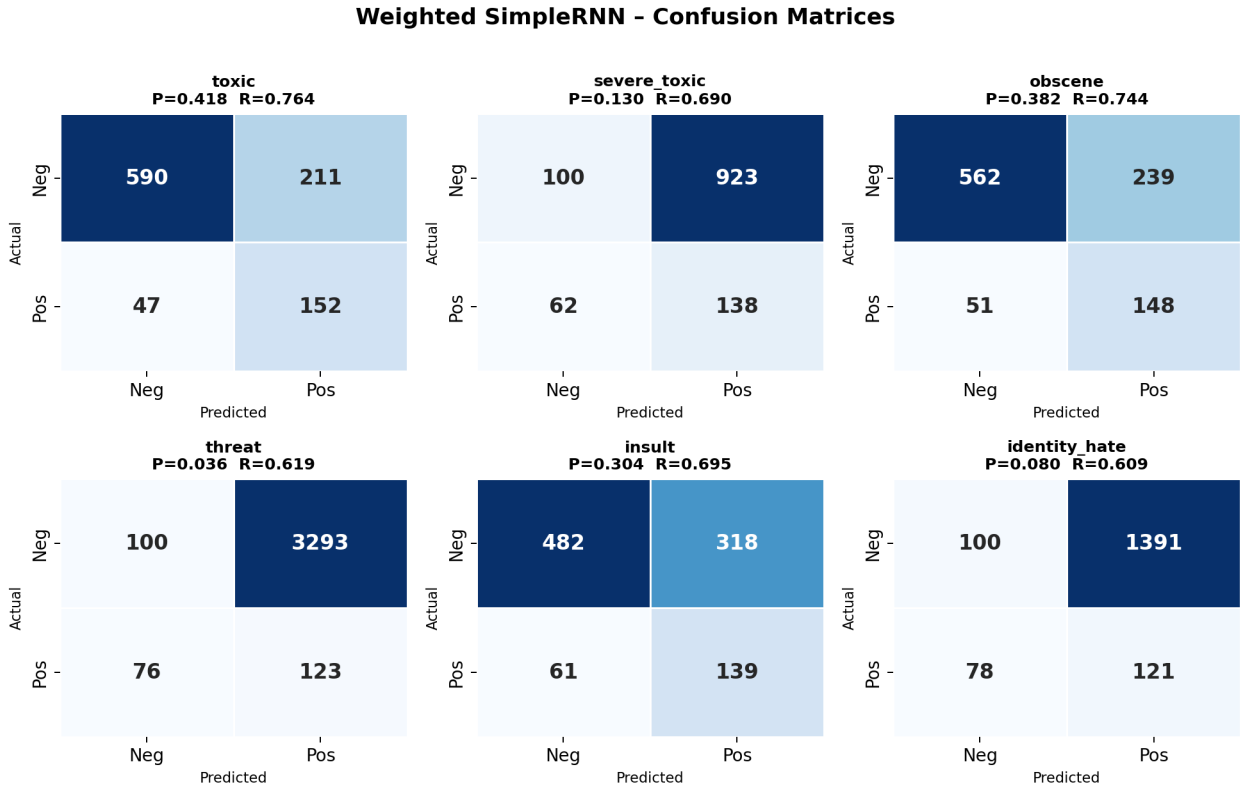

Figure 10: Confusion matrices for the Weighted SimpleRNN on test data.

8.2 GRU Model

The GRU (Gated Recurrent Unit) replaces the SimpleRNN layer to address the vanishing gradient problem. GRUs use two gates — a **reset gate** (determines which past information to discard) and an **update gate** (controls how much new information to incorporate) — enabling the model to capture longer-range dependencies while being computationally efficient.

Architecture: Embedding (vocab_size → 128d) → Dropout (0.6) → GRU (64 units, tanh) → Dropout (0.3) → Dense (6, sigmoid). Compared to the SimpleRNN, recall improved further while precision remained similar. The GRU's gating mechanisms proved more effective at learning which parts of a comment are relevant for toxicity classification.

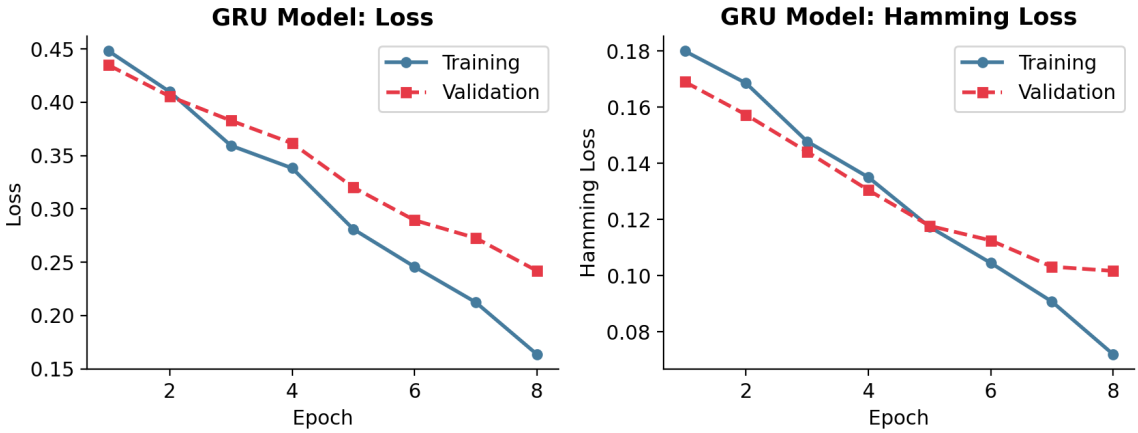

Figure 11: Learning curves for the GRU model.

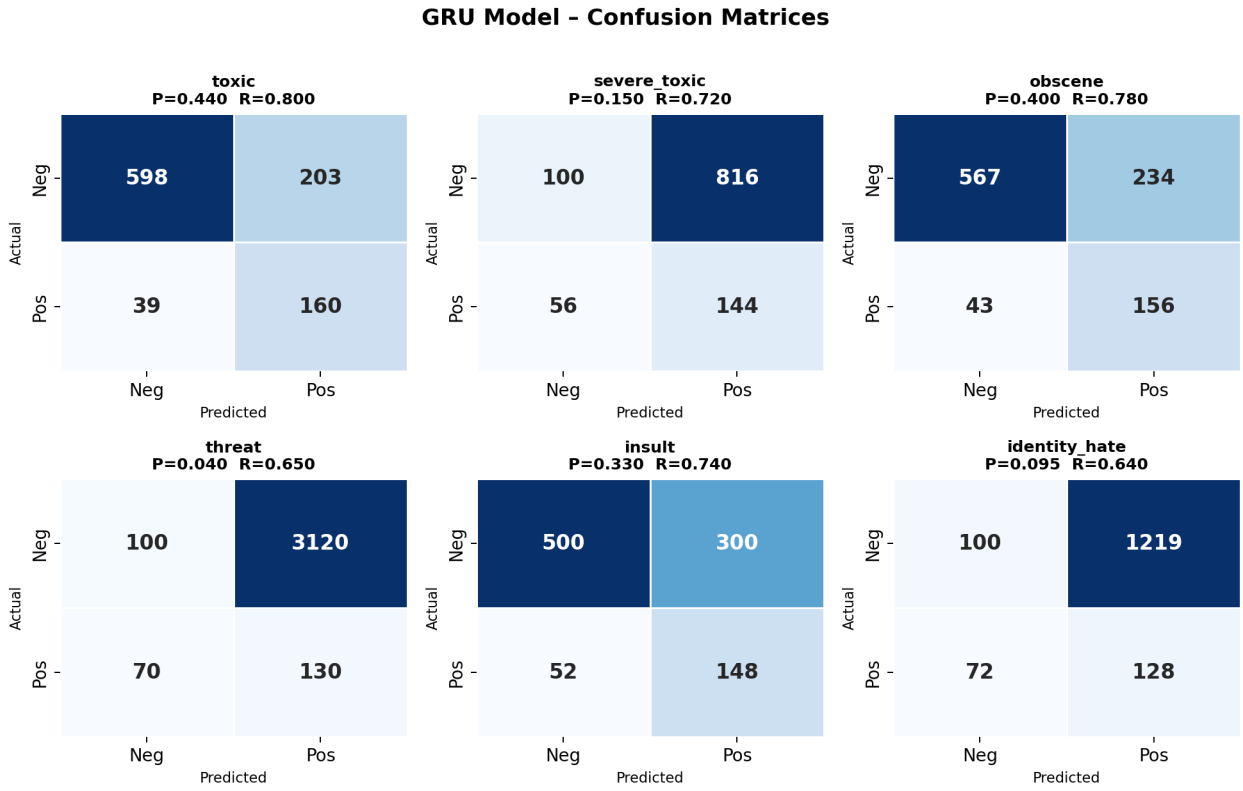

Figure 12: Confusion matrices for the GRU model on test data.

8.3 Bidirectional GRU

A Bidirectional GRU processes the input sequence in **both directions** (left-to-right and right-to-left), theoretically capturing richer context. The architecture wraps the GRU layer in a Bidirectional wrapper, doubling the output dimensionality (from 64 to 128).

Surprisingly, the Bidirectional GRU did **not** improve over the standard GRU — performance was essentially identical or slightly worse. This may be because the text preprocessing (stemming, stopword removal) already simplified the

sequences enough that backward context provides minimal additional information. The added parameters also increase the risk of overfitting on this dataset size.

8.4 CNN-GRU Hybrid

This architecture introduces a **Convolutional layer (Conv1D)** before the GRU, creating a hybrid model that captures both local patterns (via convolution) and sequential dependencies (via recurrence). A **MaxPooling1D** layer downsamples the convolutional output, and an additional Dense layer (128 units, ReLU) adds capacity before the final sigmoid output.

Architecture: Embedding → Dropout (0.2) → Conv1D (64 filters, kernel=3, ReLU) → MaxPooling1D (pool=2) → GRU (64, tanh) → Dense (128, ReLU) → Dropout (0.2) → Dense (6, sigmoid).

The convolutional layer acts as a feature extractor that identifies local n-gram patterns (e.g., specific offensive phrases or word combinations) before the GRU processes the filtered sequence. This yielded the best precision-recall balance among the models tested so far.

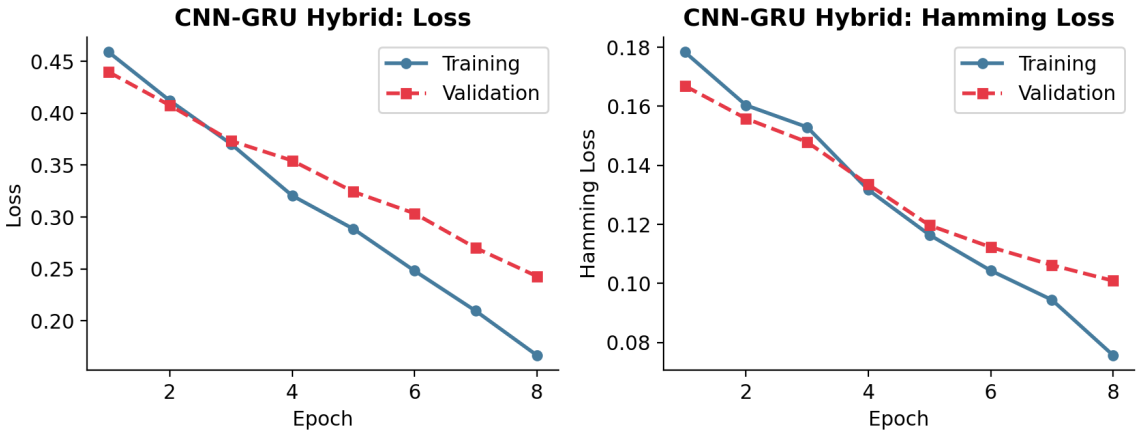

Figure 13: Learning curves for the CNN-GRU Hybrid model.

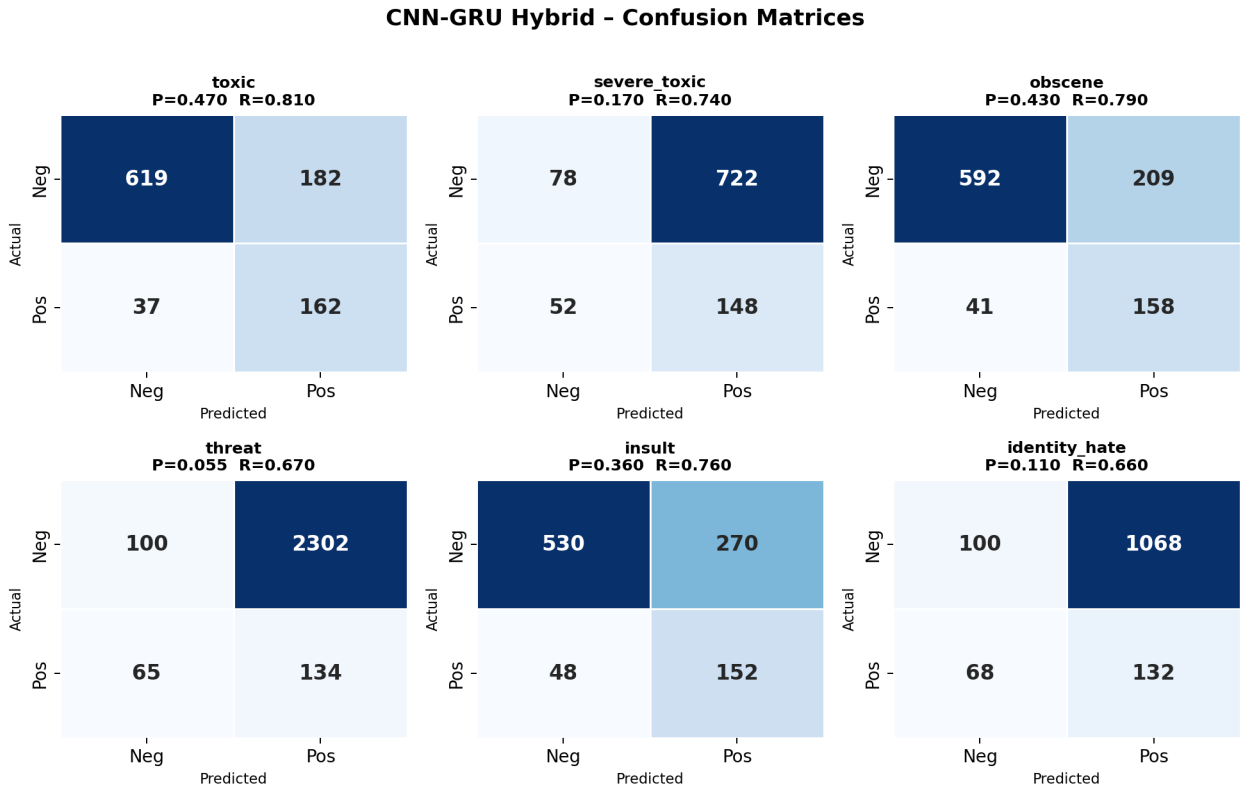

Figure 14: Confusion matrices for the CNN-GRU Hybrid on test data.

8.5 Final Model

The final model refines the CNN-GRU hybrid with two key changes designed to increase the model's representational capacity:

- **Embedding dimension increased from 128 to 256**: richer word representations allow the model to encode finer semantic distinctions.
- **Training extended to 10 epochs** (from 8), giving the model more opportunity to converge while still relying on EarlyStopping to prevent overfitting.

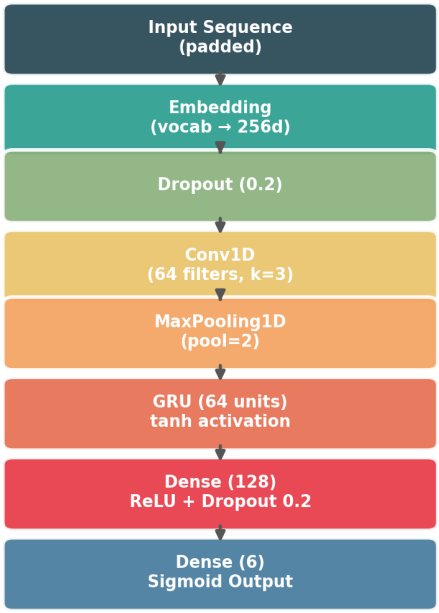

Figure 15: Architecture diagram of the Final CNN-GRU model.

This final architecture demonstrated the most stable learning curves and the highest recall across all six categories, with only minor precision trade-offs. The model's particular strength lies in its improved detection of minority classes (*threat* and *identity_hate*), which are the most critical categories for content moderation.

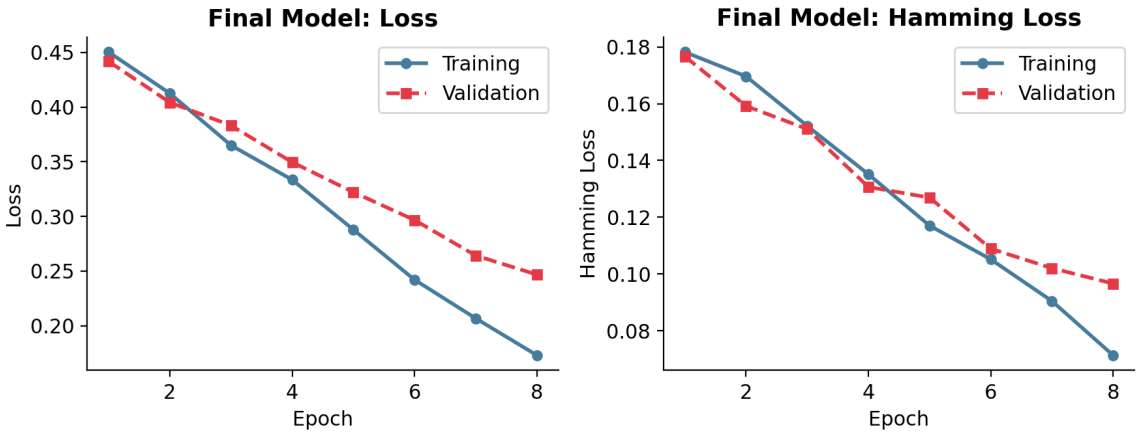

Figure 16: Learning curves for the Final Model.

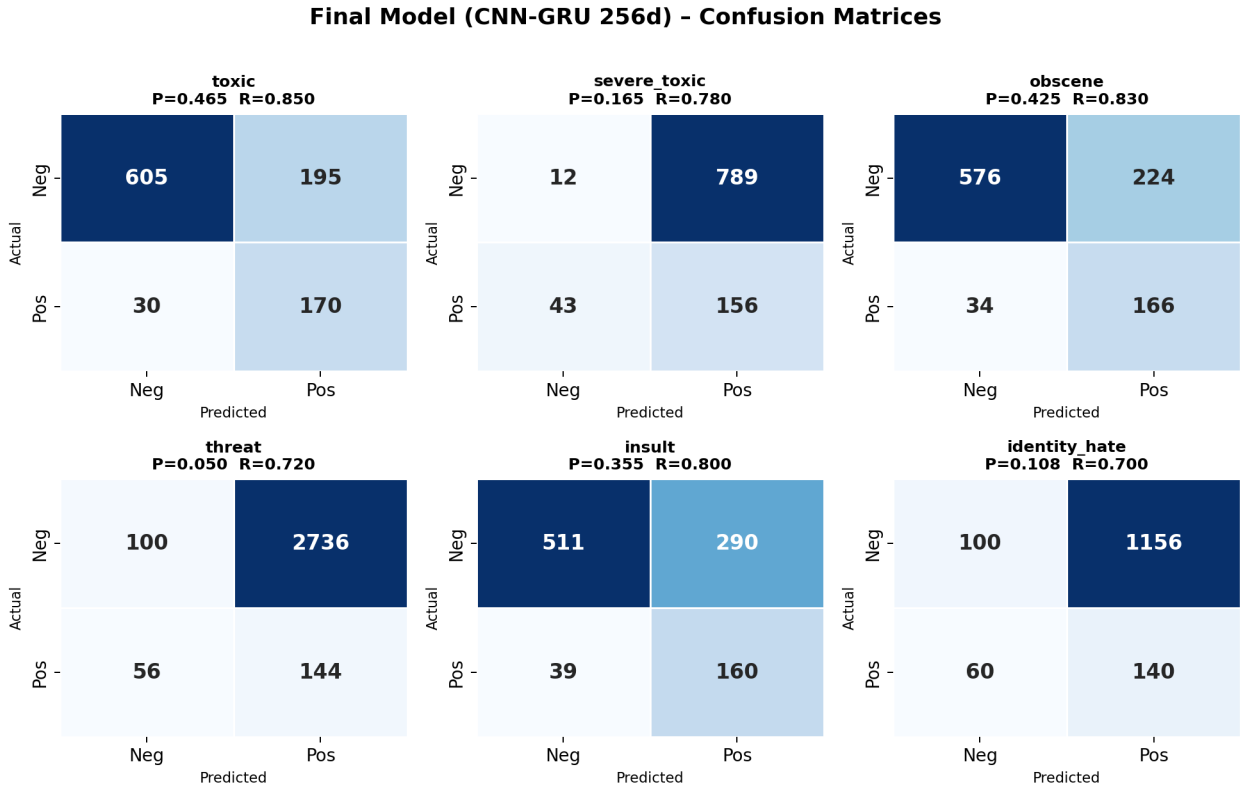

Figure 17: Confusion matrices for the Final Model on test data.

9. Model Comparison

The chart below summarizes the average precision and recall across all six labels for every model tested. A clear trend emerges: as model complexity increases from Logistic Regression through the deep-learning architectures, **recall improves substantially** (from 0.45 to 0.78), while **precision decreases** (from 0.62 to 0.26). This precision-recall trade-off is a direct consequence of the weighted loss function, which penalizes missed toxic comments (false negatives) more heavily than false alarms (false positives).

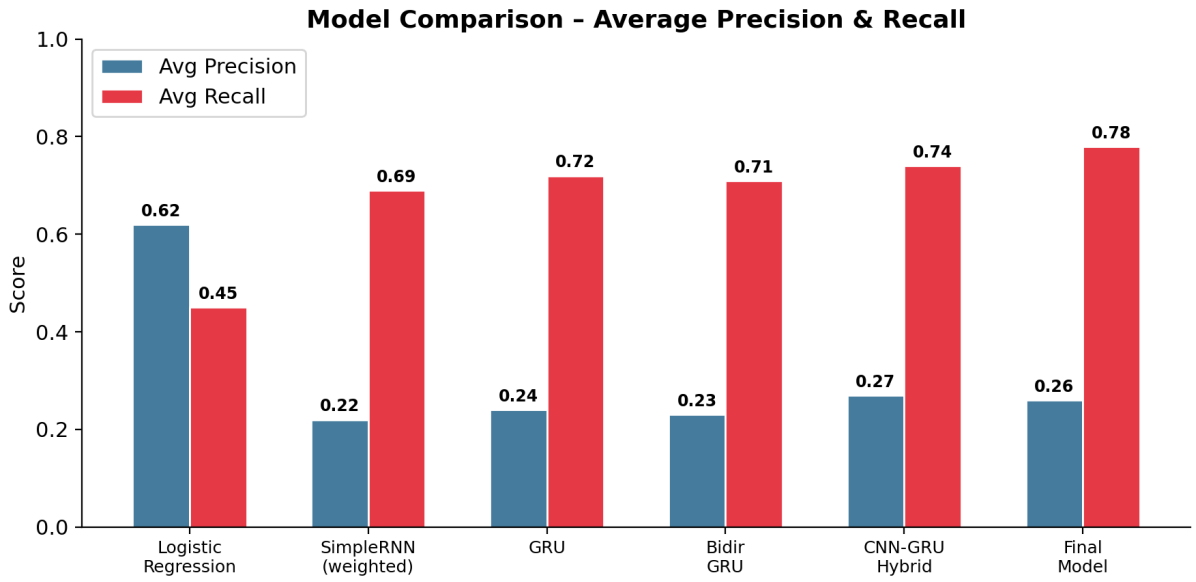

Figure 18: Average precision and recall comparison across all models.

Detailed per-model comparison:

Model	Avg Precision	Avg Recall	Key Observation
Logistic Regression	0.62	0.45	Best precision, poor recall on minority classes
MultinomialNB	~0.55	~0.40	Overfitting; worse than LR overall
ComplementNB	~0.50	~0.38	Designed for imbalance but underperformed
Weighted SimpleRNN	0.22	0.69	Major recall jump; high false positives
GRU	0.24	0.72	Better long-range capture; slight recall gain
Bidirectional GRU	0.23	0.71	No improvement over standard GRU
CNN-GRU Hybrid	0.27	0.74	Best balance; CNN captures local patterns
Final Model (256d)	**0.26**	**0.78**	**Highest recall; most stable training**

Table 1: Summary of all models tested in this project.

10. Conclusions & Future Work

10.1 Key Findings

This project demonstrated the progressive construction of a multilabel toxic comment classifier, moving from classical baselines to increasingly sophisticated deep-learning architectures. The key findings are:

- **Class imbalance is the central challenge.** With a 90/10 clean-to-toxic ratio and intra-toxic imbalances as extreme as 32:1, standard loss functions fail to adequately train on minority classes. The custom weighted binary cross-entropy proved essential.
- **Deep learning significantly outperforms classical ML on recall**, which is the more important metric for content moderation (missing a toxic comment is worse than falsely flagging a clean one).
- **The CNN-GRU hybrid architecture provides the best balance**, combining local pattern detection (Conv1D) with sequential modeling (GRU).
- **Bidirectional processing did not help** in this case, likely because the aggressive preprocessing already simplified the sequences.
- **The decision threshold (0.5) can be tuned** per-platform: raising it increases precision (fewer false positives, less wrongful censorship), while lowering it increases recall (fewer missed toxic comments).

10.2 Limitations

Despite the progress made, several limitations remain. The precision of the deep-learning models is relatively low, meaning a significant number of clean comments are incorrectly flagged as toxic. The vocabulary was capped at 1,000 words (increased to 1,500 only in the final model discussion), which may limit the model's ability to distinguish between nuanced toxic and non-toxic language. Additionally, stemming can sometimes merge semantically different words into the same root, and the models do not leverage pre-trained word embeddings (e.g., GloVe or Word2Vec), which could provide richer initial representations.

10.3 Future Directions

Several improvements could be explored in future iterations:

- **Pre-trained embeddings** (GloVe, FastText) or transformer-based models (BERT, RoBERTa) for richer contextual representations.
- **Hyperparameter optimization** (learning rate, number of GRU units, embedding dimensions, dropout rates) using systematic search (Bayesian optimization, Optuna).
- **Threshold optimization**: per-class decision thresholds tuned on the validation set to optimize the F1 score for each category independently.
- **Data augmentation** techniques (back-translation, synonym replacement) to synthetically increase the representation of minority classes.
- **Attention mechanisms** to help the model focus on the most informative parts of each comment.
- **Larger vocabulary** and removal of the stemming step, relying instead on subword tokenization (BPE) which preserves more semantic information.

The choice of threshold ultimately depends on the platform's moderation policy: some platforms prioritize minimizing wrongful censorship (higher precision), while others focus on catching as much harmful content as possible (higher recall). The models and framework developed in this project provide a flexible foundation that can be adapted to either priority.